P04 Platforms

CRE132 worksheet

What you'll make

Three kinds of platform that every platformer uses: **one-way planks** you can jump up through, a **moving platform** that carries you over a pit, and **crumbling platforms** that shake, fall away and come back.

The one-way planks need no code at all, just the right components. The other two are short scripts. On the way you'll meet a **Kinematic** Rigidbody2D, collision messages, and **coroutines**, which are a neat way to write "do this, wait, then do that".

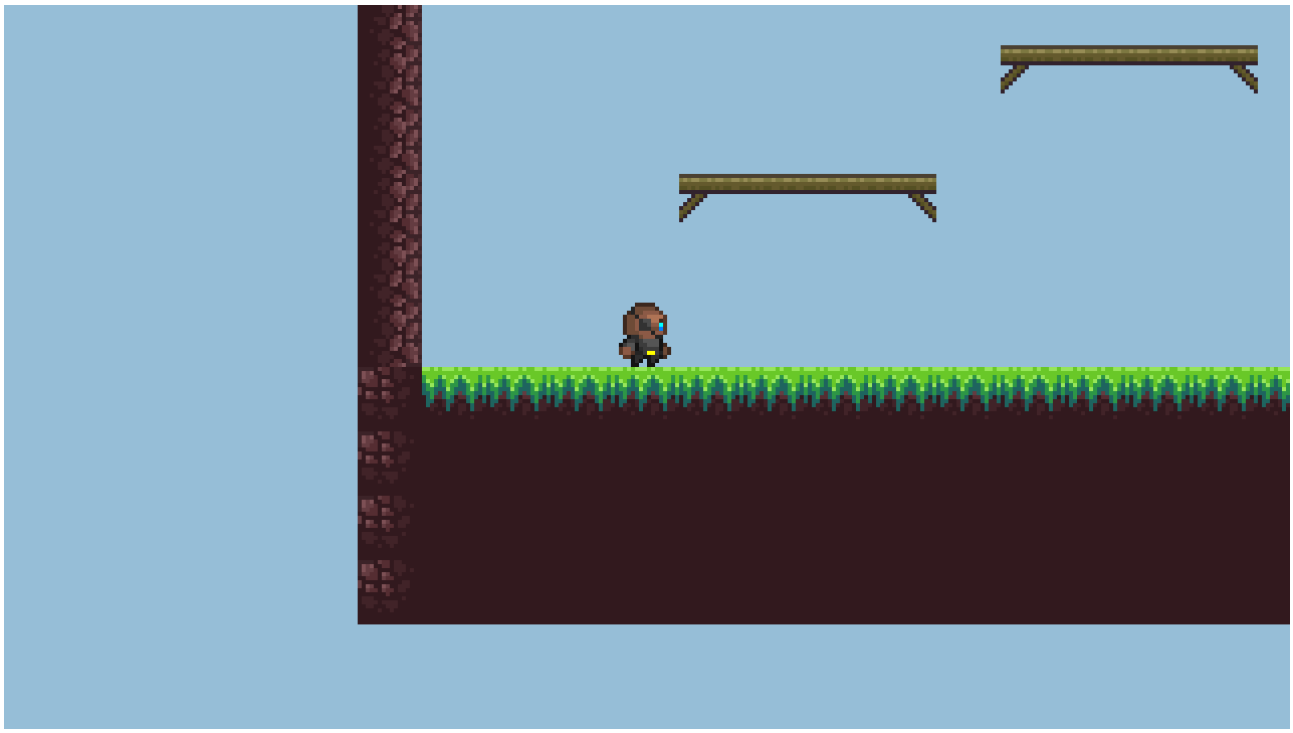


Figure 1: The finished level: one-way planks above the start, then a moving platform and crumbling platforms over two pits.

Before you start

1. Unzip `CRE132_P04_Platforms_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_P04_Platforms_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Platformer/P04_Platforms`.
3. Press **Play**. You can walk (A/D or arrows) and jump (Space) with the controller from P02, and the planks above the start already work. The **Moving Platform** over the first pit doesn't move yet, and the orange **Crumbling Platforms** over the second pit are solid for ever. Falling into a pit is safe: climb out on the right.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. One-way planks: no code

Select **One-Way Planks** (a child of **Grid**) and look at the Inspector. Nothing to add here, just see how it's built:

- A **Tilemap Collider 2D** with **Used By Effector** ticked. This hands the collider over to the effector below.
- A **Platform Effector 2D** with **Use One Way** ticked and **Surface Arc** 170. Only the top of each plank is solid: you pass through from below and the sides, and land on top. In the Scene view the effector draws its arc as a gizmo.
- The tilemap is on the **Ground** layer, so the player's ground check counts the planks as ground and you can jump off them.

Test it: press Play, stand under a plank and jump. You go straight through and land on top. Untick **Use One Way** and try again: the plank is now a solid ceiling. Tick it again before you carry on.

2. A platform that moves: MovingPlatform

Select the **Moving Platform**. It has a **Sprite Renderer**, a thin **Box Collider 2D** at the top of the plank, and a **Rigidbody 2D** with **Body Type** set to **Kinematic**. Kinematic means gravity and collisions can't push it around: only code moves it.

Make a folder `Assets/_game/Scripts/P04_Platforms` and create a script in it called `MovingPlatform`:

```
// A platform that moves back and forth between its start position and an offset,
// carrying anything standing on it.
// It needs a Kinematic Rigidbody2D: moved by this script, not pushed around by gravity
// or collisions.
using System.Collections.Generic;
using UnityEngine;

public class MovingPlatform : MonoBehaviour
{
    // Where the platform travels to, measured from where it starts (shown as a line in
    // the Scene view).
    [SerializeField] private Vector2 moveOffset = new Vector2(4f, 0f);
    [SerializeField] private float speed = 2f;

    private Rigidbody2D body;
    private Vector2 startPosition;
    private Vector2 targetPosition;

    // Everything standing on the platform right now.
    private List<Rigidbody2D> riders = new List<Rigidbody2D>();

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
        startPosition = body.position;
        targetPosition = startPosition + moveOffset;
    }

    private void FixedUpdate()
    {
        Vector2 newPosition = Vector2.MoveTowards(body.position, targetPosition,
            speed * Time.fixedDeltaTime);
        Vector2 movement = newPosition - body.position;
        body.MovePosition(newPosition);
```

```
        // Move the riders by the same amount, so they travel with the platform instead
        // of sliding off.
        foreach (Rigidbody2D rider in riders)
        {
            rider.position = rider.position + movement;
        }

        // Reached the end: head back the other way.
        if (Vector2.Distance(newPosition, targetPosition) < 0.01f)
        {
            if (targetPosition == startPosition)
            {
                targetPosition = startPosition + moveOffset;
            }
            else
            {
                targetPosition = startPosition;
            }
        }
    }

    private void OnCollisionEnter2D(Collision2D collision)
    {
        // Only things that land on top ride the platform, not things that bump into
        // its side.
        if (collision.transform.position.y > transform.position.y &&
            collision.rigidbody != null)
        {
            riders.Add(collision.rigidbody);
        }
    }

    private void OnCollisionExit2D(Collision2D collision)
    {
        if (collision.rigidbody != null)
        {
            riders.Remove(collision.rigidbody);
        }
    }

    private void OnDrawGizmos()
    {
        Vector3 start = transform.position;
        if (Application.isPlaying)
        {
            start = startPosition;
        }
        Gizmos.color = Color.yellow;
        Gizmos.DrawLine(start, start + (Vector3)moveOffset);
    }
}
```

What's going on:

- **`Vector2.MoveTowards`** gives a point a little closer to the target, never overshooting it. `speed * Time.fixedDeltaTime` is how far to go in one physics step.
- **`body.MovePosition`** is how you move a Kinematic body: physics knows it moved, so things collide with it properly.
- **The `if` at the end** swaps the target when the platform arrives, so it goes back and forth for ever.

- **The yellow gizmo line** shows the path in the Scene view, even when the game isn't running.

Add it to the **Moving Platform**. Set **Move Offset** to **X 10, Y 0** and **Speed** to **2.5**. Look in the Scene view: the yellow line should reach across the pit to the island.

Test it: press Play and jump onto the platform. It moves.

3. Carrying the player

Moving the platform is only half the job. The player is a separate physics body, so on its own it stays where it is and the platform slides away underneath it. That's what the `riders` list in your script is for:

- **`OnCollisionEnter2D`** runs once when something touches the platform. If it is **above** the platform and has a Rigidbody2D, it goes in the `riders` list. Something bumping into the side isn't added.
- **`OnCollisionExit2D`** runs when it leaves, and takes it out of the list again.
- **In `FixedUpdate`**, `movement` is how far the platform moved this step. The `foreach` loop moves every rider by the same amount, so they travel with it.

Test it: stand still on the platform and let go of the keys. You should be carried all the way across. Then walk off the end and check that you aren't dragged along in the air.

4. Platforms that fall: CrumblingPlatform

Open one of the three **Crumbling Platforms**. The parent object has the **Box Collider 2D**; the orange sprite is on a child called **Visual**. That matters: when we shake the sprite, the collider stays put, so the player isn't shaken about.

Create `CrumblingPlatform` in the `P04_Platforms` folder:

```
// A platform that shakes when the player lands on it, falls away, and comes back a few
// seconds later.
using System.Collections;
using UnityEngine;

public class CrumblingPlatform : MonoBehaviour
{
    [SerializeField] private float shakeTime = 0.5f;
    [SerializeField] private float shakeAmount = 0.05f;
    [SerializeField] private float respawnTime = 3f;

    // The sprite is a child object, so it can shake without moving the collider.
    [SerializeField] private Transform visual;

    private Collider2D platformCollider;
    private SpriteRenderer spriteRenderer;
    private Vector3 visualStartPosition;
    private bool isCrumbling;

    private void Awake()
    {
        platformCollider = GetComponent<Collider2D>();
        spriteRenderer = visual.GetComponent<SpriteRenderer>();
        visualStartPosition = visual.localPosition;
    }

    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (!isCrumbling && collision.gameObject.CompareTag("Player") &&
            collision.transform.position.y > transform.position.y)
```

```
        {
            StartCoroutine(Crumble());
        }
    }

    private IEnumerator Crumble()
    {
        isCrumbling = true;

        // Shake: a small random offset every frame for shakeTime seconds.
        float timer = 0f;
        while (timer < shakeTime)
        {
            visual.localPosition = visualStartPosition +
                (Vector3)(Random.insideUnitCircle * shakeAmount);
            timer = timer + Time.deltaTime;
            yield return null;
        }
        visual.localPosition = visualStartPosition;

        // Gone: no collider, no sprite.
        platformCollider.enabled = false;
        spriteRenderer.enabled = false;

        yield return new WaitForSeconds(respawnTime);

        platformCollider.enabled = true;
        spriteRenderer.enabled = true;
        isCrumbling = false;
    }
}
```

- **A coroutine** (`IEnumerator Crumble`, started with `StartCoroutine`) is a method that can pause. **`yield return null`** waits one frame; **`yield return new WaitForSeconds(respawnTime)`** waits that many seconds. The rest of the game keeps running meanwhile.
- **`isCrumbling`** stops a second crumble starting while the first one is still going.
- **`CompareTag("Player")`** and the height check mean only the player landing on top sets it off, not bumping it from below.
- **`Random.insideUnitCircle`** is a random point in a circle of size 1; times `shakeAmount` it becomes a tiny wobble.
- **Switching off** the collider and the sprite makes the platform vanish, without destroying it, so it can come back.

Add it: select all three **Crumbling Platforms** and add **CrumblingPlatform**. Leave **Shake Time** 0.5, **Shake Amount** 0.05 and **Respawn Time** 3. Then, **one platform at a time**, drag its own child **Visual** into its **Visual** slot.

Test it: land on a crumbling platform. It shakes for half a second, drops you, and comes back three seconds later.

Check your work

- ☐ You can jump up through the planks and stand on them.
- ☐ The moving platform goes back and forth across the first pit and carries you when you stand still on it.
- ☐ Walking into the side of the moving platform doesn't make you ride it.
- ☐ Each crumbling platform shakes, falls away after half a second and comes back after three.

- ☐ Compare with `CRE132_P04_Platforms_Finished` if something behaves differently.

Key points

- **Platform Effector 2D** gives one-way platforms with no code. Remember **Used By Effector** on the collider.
- **Things that move and carry other things** need a **Kinematic** Rigidbody2D, moved with `MovePosition`. The riders have to be moved too.
- **Coroutines** are good for "do this, wait, do that" sequences, like shake, vanish, come back.

If something goes wrong

- **The planks are solid from below: Use One Way** is unticked on the Platform Effector 2D, or **Used By Effector** is unticked on the Tilemap Collider 2D.
- **The moving platform falls off the screen:** its Rigidbody 2D **Body Type** is Dynamic, not Kinematic.
- **The moving platform doesn't move: Move Offset** is still 0, or **Speed** is 0.
- **NullReferenceException in CrumblingPlatform's Awake:** a **Visual** slot is empty. Check all three platforms.
- **The wrong platform shakes:** you dragged another platform's Visual into the slot. Each one needs its own child.
- **Nothing happens when you land on a crumbling platform:** the player must have the **Player** tag (top of the Inspector).

Try it

1. Make a lift: duplicate the **Moving Platform**, put it somewhere with space above, and set **Move Offset** to **(0, 5)**.
2. Drop down through a one-way plank by pressing Down and Jump together. (Hint: `Physics2D.IgnoreCollision` between the player's collider and the plank's collider, for a moment, then switch it back.)
3. Make a crumbling platform warn the player by flashing red before it falls. (Hint: change `spriteRenderer.color` inside the shake loop, and put it back afterwards.)